INSTITUTO POLITÉCNICO NACIONAL

ESCUELA SUPERIOR DE CÓMPUTO

Servicios Web: Arquitectura de Servicios y Concurrencia Optimista

Presenta

Aguilar Souza Iker Itzae

Grupo

7CM1

Materia

Sistemas Distribuidos

Ciudad de México, 21 de octubre de 2026

1. Antecedentes

Un **servicio web** es un componente de software accesible a través de la red mediante protocolos y formatos estandarizados, diseñado para ser consumido por otros programas y no por personas. Su rasgo definitorio no es la tecnología que emplea internamente, sino que su interfaz está **descrita de forma que cualquier cliente pueda invocarla** con independencia del lenguaje o la plataforma en que esté escrito.

1.1. La pila de servicios web

La arquitectura de servicios se organiza en capas, cada una apoyada en la inferior. De abajo hacia arriba:

- **Comunicaciones.** El transporte. En la práctica totalidad de los casos, **HTTP** sobre TCP/IP. Aporta direccionamiento universal mediante URL, atraviesa cortafuegos y dispone de una semántica de métodos y códigos de estado ya definida.
- **Mensajes.** El formato de la invocación y de su respuesta. Aquí conviven dos tradiciones: **SOAP**, que encapsula todo en un sobre XML y usa HTTP como mero túnel; y el estilo **REST**, que aprovecha los propios verbos de HTTP (`GET`, `POST`, `PUT`, `DELETE`) sobre recursos identificados por URL.
- **Descripción.** El contrato: qué operaciones ofrece el servicio, con qué parámetros y qué devuelve. En el mundo SOAP es el **WSDL** (XML); en el mundo REST, **OpenAPI** con esquemas JSON.
- **Procesos.** La capa superior, donde los servicios dejan de considerarse aislados: **descubrimiento** (localizar un servicio sin conocer su dirección), **agregación** (combinar varios en una operación de mayor nivel) y **coreografía** (la secuencia de interacciones que conforma un proceso de negocio).

Dos preocupaciones atraviesan verticalmente todas las capas:

- **Seguridad:** autenticación, autorización y cifrado. Afecta al transporte (HTTPS), a los mensajes (firmas) y a los procesos (quién puede invocar qué).
- **Gestión:** supervisión del estado de los servicios, registro de actividad y métricas. Sin ella un sistema distribuido es inoperable, porque los fallos dejan de ser evidentes.

1.2. SOAP frente a REST

	SOAP	**REST**
Unidad de diseño	Operación	Recurso
Endpoints	Uno solo	Uno por recurso
Formato	XML obligatorio	JSON habitualmente
Verbos de HTTP	Siempre `POST`	`GET`, `POST`, `PUT`, `DELETE`
Contrato	WSDL, formal	OpenAPI, descriptivo
Errores	`Fault` en el cuerpo	Códigos de estado de HTTP

La diferencia de fondo es que SOAP trata a HTTP como un **medio de transporte** —da igual que debajo haya HTTP, SMTP o una cola de mensajes— mientras que REST lo trata como un **modelo de aplicación** y se apoya en su semántica.

1.3. Ausencia de estado y concurrencia

HTTP es un protocolo **sin estado**: cada petición es independiente y el servidor no está obligado a recordar nada entre una y otra. Esta propiedad es la que permite escalar horizontalmente, pero plantea un problema para operaciones que en las prácticas anteriores se resolvían reteniendo estado entre mensajes.

En la práctica 3, un cliente «entraba» a la sección crítica y el servidor recordaba que era el titular hasta que la liberase. Con servicios web no hay conexión persistente ni sesión que sostenga esa titularidad. Caben dos soluciones:

- **Control pesimista:** reproducir el bloqueo mediante un recurso de tipo *lease*, que se adquiere, se renueva y expira. Funciona, pero reintroduce estado en el servidor y contradice el estilo.
- **Control optimista:** no impedir el conflicto, sino **detectarlo**. El servidor asigna a cada recurso una etiqueta de versión (`ETag`) que entrega al leerlo; el cliente la devuelve en la cabecera `If-Match` al escribir, y si entretanto el recurso cambió, la escritura se rechaza.

El segundo es el mecanismo propio de HTTP, definido en el RFC 7232, y se apoya en dos códigos de estado: **412 Precondition Failed** cuando la precondición no se cumple, y **428 Precondition Required** cuando el servidor exige que se envíe. Se añade **409 Conflict** para los casos en que la operación es imposible por el estado del recurso, aunque la versión coincida.

La diferencia filosófica con las prácticas anteriores es notable. El control pesimista **impide** que el conflicto ocurra, al precio de serializar los accesos; el optimista **deja que ocurra** y lo detecta después, apostando a que los conflictos son raros. Cuál conviene depende de la probabilidad de colisión, que a su vez depende de la duración de la ventana entre la lectura y la escritura.

2. Planteamiento del Problema

Las cinco prácticas anteriores construyeron, pieza a pieza, un sistema distribuido completo: sockets, protocolo de aplicación propio, exclusión mutua centralizada, exclusión mutua distribuida y un middleware de objetos remotos. Todo ello con mecanismos **construidos a medida**, que funcionan pero que solo entienden los programas escritos para ellos.

El problema que se aborda ahora es **reconstruir el sistema sobre estándares abiertos**, de modo que cualquier cliente —un navegador, una aplicación móvil, un servicio ajeno— pueda consumirlo sin conocer los detalles internos. Esto implica recorrer todas las capas de la pila de servicios y, en particular, enfrentar la consecuencia de adoptar HTTP: la pérdida del estado entre peticiones.

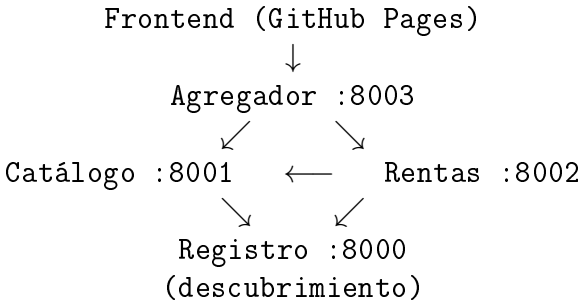
Los requisitos son:

- **Varios servicios independientes** que se descubran, se compongan y colaboren en un proceso de negocio, cubriendo la capa de Procesos.
- **Dos estilos de mensajería** sobre el mismo recurso, REST y SOAP, para poder contrastarlos.
- **Descripción formal** del servicio en ambos estilos: OpenAPI y WSDL.
- **Concurrencia sin bloqueo:** resolver mediante `ETag` e `If-Match` el mismo problema que las prácticas 3 y 4 resolvían con cerrojos, y medir la diferencia.
- **Seguridad y gestión** presentes en todos los servicios.
- **Despliegue público** desde un repositorio de GitHub, con el servicio en una plataforma de nube y el cliente web en GitHub Pages.

3. Solución

El sistema se compone de cuatro servicios independientes, un módulo común y un cliente web.

3.1. Arquitectura



Servicio	Puerto	Responsabilidad
`registro.py`	8000	Descubrimiento: traduce nombres en URL
`catalogo.py`	8001	Catálogo. REST con `ETag` y endpoint SOAP
`rentas.py`	8002	Rentas. Compone el catálogo y reintenta
`agregador.py`	8003	Agregación y coreografía

El módulo `comun.py` reúne lo transversal: resolución de direcciones por variable de entorno, CORS, verificación de la clave de API, cálculo de `ETag` y el cliente HTTP entre servicios.

3.2. Correspondencia con la pila

Capa	Dónde se implementa
Procesos	`registro.py` y `agregador.py`
Descripción	OpenAPI en `/docs`, WSDL en `/servicio.wsdl`
Mensajes	Endpoints REST y `/soap` de `catalogo.py`
Comunicaciones	HTTP mediante FastAPI y uvicorn
Seguridad	`exigir_clave()`, cabecera `X-API-Key`, HTTPS al desplegar
Gestión	`/health` y `/metricas` en los cuatro servicios

3.3. Elección de FastAPI

Se eligió FastAPI porque **genera la capa de Descripción automáticamente**: a partir de las anotaciones de tipo y los modelos de datos produce un documento OpenAPI y una interfaz navegable en `/docs`. El contrato deja de ser documentación que se escribe aparte y se desactualiza, y pasa a derivarse del propio código. Es exactamente el papel que el WSDL cumple en SOAP, con la diferencia de que allí el contrato suele escribirse primero y el código generarse después.

4. Implementación

4.1. Capa de mensajes: ETag y verificación de precondiciones

Listing 1: Cálculo de la versión y comprobación (comun.py)

```
def calcular_etag(estado) -> str:
    """Huella del estado actual del recurso."""
    datos = json.dumps(estado, sort_keys=True, ensure_ascii=False)
    return '"' + hashlib.sha256(datos.encode("utf-8")).hexdigest()[:16] +
        '"'


def verificar_if_match(if_match: str | None, etag_actual: str):
    """Precondiciones de HTTP: 428 si falta, 412 si no coincide."""
    if if_match is None:
        raise HTTPException(
            status_code=428,
            detail="Se requiere la cabecera If-Match con el ETag leido")
    if if_match != etag_actual and if_match != "*":
        raise HTTPException(
            status_code=412,
            detail=f"El recurso cambio desde su lectura (ETag actual {
               etag_actual})")
```

Explicación: el `ETag` es una huella del contenido, no un número de secuencia arbitrario: dos estados idénticos producen la misma etiqueta. Se ordenan las claves antes de serializar para que la huella no dependa del orden del diccionario. La distinción entre **428** y **412** es deliberada: el primero indica que el cliente *no envió* la precondición —un error de programación

del cliente—, mientras que el segundo indica que la envió pero *ya no es válida*, que es la situación de carrera legítima.

4.2. Operación de escritura con control optimista

Listing 2: Renta de una película (catalogo.py)

```python
@app.post("/catalogo/{titulo}/rentar", tags=["catalogo (REST)"],
          dependencies=[Depends(exigir_clave)])
def rentar(titulo: str, solicitud: SolicitudRenta, response: Response,
           if_match: str = Header(default=None, alias="If-Match")):
    metricas.peticiones += 1
    with _candado:
        pelicula = _buscar(titulo)

        # Concurrencia optimista: no se bloquea, se comprueba
        try:
            verificar_if_match(if_match, pelicula.etag())
        except HTTPException:
            metricas.conflictos += 1
            raise

        if not pelicula.disponible:
            metricas.conflictos += 1
            raise HTTPException(
                status_code=409,
                detail=f"'{titulo}' ya esta rentada por el cliente {
                    pelicula.rentada_por}")

        pelicula.disponible = False
        pelicula.rentada_por = solicitud.cliente
        pelicula.version += 1
        etag = pelicula.etag()

    response.headers["ETag"] = etag
    return {"mensaje": f"Renta de '{titulo}' concedida al cliente {
        solicitud.cliente}",
            "pelicula": pelicula.como_dict()}
```

Explicación: el cerrojo `_candado` protege únicamente la estructura de datos **dentro del proceso**, durante unos microsegundos; no es el mecanismo de coordinación entre clientes. La coordinación la hace la comprobación del `ETag`, que no retiene nada entre peticiones. Obsérvese que la respuesta incluye el **nuevo** `ETag`: así el cliente que acaba de escribir puede encadenar otra operación sin releer. La versión se incrementa en cada cambio, lo que garantiza que la huella sea distinta aunque el estado anterior se repita.

4.3. Composición de servicios con reintento

Listing 3: El servicio de rentas absorbe los conflictos (rentas.py)

```python
for intento in range(1, MAXIMO_REINTENTOS + 1):
    # 1. Lectura: obtener el estado actual y su ETag
```

```
    lectura = catalogo.get(f"/catalogo/{solicitud.titulo}")
    etag = lectura.headers.get("ETag")

    if not lectura.json().get("disponible"):
        raise HTTPException(status_code=409,
                            detail=f"'{solicitud.titulo}' no esta
                               disponible")

    # 2. Escritura condicionada al ETag leido
    escritura = catalogo.post(
        f"/catalogo/{solicitud.titulo}/rentar",
        json={"cliente": solicitud.cliente},
        headers=_cabeceras(etag))

    if escritura.status_code == 200:
        ...
        return {"mensaje": "Renta registrada", "renta": registro, "
           intentos": intento}

    if escritura.status_code in (409, 412):
        # Otro cliente modifico el recurso: se vuelve a leer y reintentar
        reintentos += 1
        metricas.conflictos += 1
        continue
```

Explicación: este es el patrón completo de la concurrencia optimista, **leer–modificar–escribir con reintento**. El bucle es imprescindible: sin él, la detección del conflicto solo sirve para informar del fracaso. Nótese que el reintento vuelve a **leer**, porque reenviar el mismo `ETag` caducado fallaría de nuevo indefinidamente. El límite de intentos evita que dos clientes en competencia perpetua consuman el servicio. Se distingue además entre el **409 semántico** que se lanza antes de intentar —la película no está disponible, y reintentar no lo cambiaría— y el **412 técnico**, que sí merece otro intento.

4.4. Capa de procesos: coreografía

Listing 4: Secuencia completa del proceso de negocio (agregador.py)

```
# Paso 1: consultar el catalogo
respuesta = catalogo.get("/catalogo")
peliculas = respuesta.json()["peliculas"]
pasos.append({"paso": 1, "accion": "GET catalogo/catalogo",
              "resultado": f"{len(peliculas)} peliculas, ETag {respuesta.
                 headers.get('ETag')}"})

# Paso 2: elegir una disponible
disponibles = [p["titulo"] for p in peliculas if p["disponible"]]
elegida = disponibles[0]

# Paso 3: delegar la renta en el servicio correspondiente
creacion = rentas.post("/rentas",
                       json={"cliente": solicitud.cliente, "titulo":
                          elegida},
                       headers={"X-API-Key": CLAVE_API})
```

```
15
16 # Paso 4: confirmar contra el catalogo
17 confirmacion = catalogo.get(f"/catalogo/{elegida}").json()
```

Explicación: el agregador no posee ningún dato: su valor está en **conocer el orden de las invocaciones**. Cada paso se registra para que la traza del proceso sea observable, que es justamente lo que la capa de Procesos pretende describir. Nótese que el agregador llama al servicio de *rentas* y no directamente al catálogo: respeta las fronteras entre servicios en lugar de saltárselas, de modo que la lógica de reintento vive en un solo lugar.

4.5. Capa de mensajes: el contraste con SOAP

Listing 5: Endpoint único y errores dentro del cuerpo (catalogo.py)

```
1  def _fault(codigo: str, mensaje: str) -> Response:
2      """En SOAP el error viaja DENTRO del cuerpo, con codigo HTTP 200."""
3      cuerpo = (f'    <soap:Fault>\n'
4                f'      <faultcode>{codigo}</faultcode>\n'
5                f'      <faultstring>{escape(mensaje)}</faultstring>\n'
6                f'    </soap:Fault>')
7      return _sobre(cuerpo)
8
9
10 @app.post("/soap", tags=["catalogo (SOAP)"])
11 async def punto_soap(request: Request):
12     """Acepta ConsultarCatalogo y Rentar dentro de un sobre SOAP."""
13     crudo = await request.body()
14     arbol = ET.fromstring(crudo)
15     cuerpo = arbol.find(f"{{{SOAP_ENV}}}Body")
16     operacion = cuerpo[0]
17     nombre = operacion.tag.split("}")[-1]
18     ...
```

Explicación: todas las operaciones SOAP llegan al **mismo endpoint** mediante `POST`, y cuál se invoca se determina leyendo el primer elemento del cuerpo del sobre. La URL deja de identificar un recurso y pasa a ser una simple dirección de buzón. La consecuencia más visible es el tratamiento de los errores: un `Fault` se devuelve con código HTTP **200**, porque desde el punto de vista de HTTP el mensaje se entregó correctamente; el error pertenece a la capa de aplicación, no a la de transporte.

4.6. Capas transversales

Listing 6: Seguridad y gestión (comun.py y rentas.py)

```
1  def exigir_clave(x_api_key: str = Header(default=None)):
2      """Dependencia de FastAPI: protege las operaciones de escritura."""
3      if x_api_key != CLAVE_API:
4          raise HTTPException(status_code=401,
5                              detail="Falta la cabecera X-API-Key o su valor
                                   es incorrecto")
6      return x_api_key
```

```python


@app.get("/health", tags=["gestion"])
def salud():
    """Un servicio no esta sano si su dependencia no responde."""
    try:
        dependencia = catalogo.get("/health").status_code == 200
    except Exception:
        dependencia = False
    return {"estado": "vivo" if dependencia else "degradado",
            "servicio": "rentas", "dependencia_catalogo": dependencia}
```

Explicación: la seguridad se expresa como una **dependencia** que se declara en cada endpoint que la necesita, de modo que las lecturas quedan abiertas y las escrituras protegidas sin repetir código. La comprobación de salud es **transitiva**: un servicio cuya dependencia no responde se declara *degradado* aunque él mismo funcione, porque en una arquitectura compuesta la salud propia no basta para prestar el servicio.

4.7. Reutilización de conexiones

Listing 7: Cliente HTTP persistente entre servicios (comun.py)

```python
class ClienteServicio:
    """Mantiene la conexion abierta entre invocaciones (keep-alive)."""

    def __init__(self, servicio: str, tiempo_limite: float = 10.0):
        self.servicio = servicio
        self._http = httpx.Client(
            timeout=tiempo_limite,
            limits=httpx.Limits(max_keepalive_connections=20,
               max_connections=50),
        )
```

Explicación: la primera versión abría un socket nuevo en cada invocación entre servicios. Funcionaba, pero la latencia media de una renta era de **3,477 s**; con el cliente persistente bajó a **0,018 s**, casi doscientas veces menos. El coste de establecer una conexión TCP —el saludo de tres vías estudiado en la práctica 2— se paga una sola vez en lugar de en cada llamada. En una arquitectura de servicios, donde una petición del usuario desencadena varias invocaciones internas, ese coste se multiplica.

5. Resultados

5.1. Verificación de las capas

Los cuatro servicios se arrancan en orden y se registran automáticamente:

Listing 8: Descubrimiento y salud compuesta

```
$ curl http://127.0.0.1:8003/health
{"estado":"vivo","servicio":"agregador",
 "dependencias":{"catalogo":true,"rentas":true,"registro":true}}
```

```
$ curl http://127.0.0.1:8003/servicios
{"servicios":[
  {"nombre":"rentas",   "url":"http://127.0.0.1:8002"},
  {"nombre":"catalogo", "url":"http://127.0.0.1:8001"},
  {"nombre":"agregador","url":"http://127.0.0.1:8003"}]}
```

El control de concurrencia responde con el código adecuado en cada situación:

Listing 9: Secuencia completa de códigos de estado

```
1. GET /catalogo                        -> 200  ETag: "ef18d834ca4c77ad"
2. GET /catalogo/The Matrix             -> 200  ETag: "2a7cce75dc62ea21"
3. POST .../rentar  sin If-Match        -> 428  Precondition Required
4. POST .../rentar  sin X-API-Key       -> 401  Unauthorized
5. POST .../rentar  con If-Match        -> 200  {"mensaje":"Renta concedida...",
                                                 "version":2}
6. POST .../rentar  con el MISMO ETag -> 412  "El recurso cambio desde su lectura
                                                 (ETag actual \"70179933061ef35d\")"
```

El paso 6 es el núcleo del mecanismo: el `ETag` que era válido en el paso 5 ha dejado de serlo precisamente **porque** la operación anterior tuvo éxito. Un segundo cliente que hubiera leído a la vez que el primero recibiría ese mismo 412.

5.2. Contraste entre REST y SOAP

Listing 10: La misma operación fallida en los dos estilos

```
--- REST ---
POST /catalogo/The%20Matrix/rentar   (pelicula ya rentada)
--> HTTP 409 Conflict
    {"detail":"'The Matrix' ya esta rentada por el cliente 3"}

--- SOAP ---
POST /soap
  <soap:Envelope><soap:Body>
    <Rentar xmlns="http://escom.ipn.mx/blockbuster">
      <Titulo>The Matrix</Titulo><Cliente>5</Cliente>
    </Rentar>
  </soap:Body></soap:Envelope>
--> HTTP 200 OK          <-- el transporte considera que todo fue bien
    <soap:Envelope><soap:Body>
      <soap:Fault>
        <faultcode>soap:Server</faultcode>
        <faultstring>'The Matrix' ya esta rentada</faultstring>
      </soap:Fault>
    </soap:Body></soap:Envelope>
```

La diferencia es ilustrativa. En REST el error se expresa en el **protocolo**, de modo que cualquier intermediario —un proxy, una caché, un balanceador— puede reaccionar ante un 409 sin interpretar el cuerpo. En SOAP el error está **oculto dentro del mensaje**: para un intermediario, la petición fue un éxito. Esa opacidad es el precio de la independencia respecto al transporte.

5.3. Experimento de concurrencia optimista

Se ejecutaron seis clientes concurrentes, cinco ciclos cada uno, sobre un catálogo de tres películas, con una ventana media de 0,3 s entre la lectura del `ETag` y la escritura. Se compararon dos configuraciones idénticas salvo en un punto: si los conflictos se reintentan o no.

	Sin reintento (cliente → catálogo)	**Con reintento** (cliente → rentas → catálogo)
Operaciones solicitadas	25	28
Rentas concedidas	19	21
Conflictos vistos por el cliente	6	7
de ellos, 412 (técnicos)	**6**	**0**
de ellos, 409 (semánticos)	0	7
Reintentos internos del servicio	—	8
Intentos por renta concedida	1,32	1,38
Latencia media	0,193 s	0,281 s

El resultado decisivo está en el desglose de los conflictos. **Sin reintento, los seis conflictos son 412**: fallos puramente técnicos provocados por la carrera, en los que la película podía estar perfectamente disponible y el usuario recibe un error que no le dice nada útil. **Con reintento, el cliente no ve ni un solo 412**: los ocho que se produjeron fueron absorbidos por el servicio de rentas, que releyó y volvió a intentarlo. Los siete rechazos que quedan son 409 **legítimos**: la película estaba realmente rentada, y ningún reintento lo cambiaría.

El precio es la latencia, que sube de 0,193 s a 0,281 s, un 45 % más. Es el coste de las lecturas y escrituras repetidas. A cambio, el conflicto deja de ser problema del cliente y se convierte en un detalle de implementación del servicio.

5.4. Sobre el tamaño de la ventana

Una observación surgida durante las pruebas merece mención. Antes de introducir el cliente HTTP persistente, la latencia era de 3,477 s y la tasa de conflicto rondaba el 50 %. Al reducir la latencia a 0,018 s, los conflictos **desaparecieron casi por completo**, hasta el punto de que hubo que añadir una ventana artificial configurable para poder medirlos.

Esto confirma de forma empírica el principio que gobierna la concurrencia optimista: **la probabilidad de conflicto es proporcional a la duración de la ventana** entre la lectura y la escritura. De ahí que el enfoque optimista sea adecuado cuando las operaciones son breves y las colisiones improbables, y que el pesimista de las prácticas 3 y 4 lo sea cuando la sección crítica es larga o la contención alta. La elección no es cuestión de gusto, sino una consecuencia medible de los tiempos del sistema.

5.5. Despliegue

El repositorio contiene todo lo necesario para desplegarse:

- `render.yaml` declara los cuatro servicios e inyecta en cada uno las URL de los demás mediante variables de entorno, de forma que el mismo código funciona en local y en la nube sin modificarse.
- `docs/index.html` es el cliente web, publicable en GitHub Pages.
- Las cabeceras CORS se verificaron con un origen distinto, incluida `Access-Control-Expose-Headers: ETag`, sin la cual el navegador recibe la cabecera pero **JavaScript no puede leerla**, lo que rompería el mecanismo completo.

Listing 11: Verificación de CORS desde otro origen

```
$ curl -X OPTIONS .../catalogo/The%20Matrix/rentar \
       -H "Origin: https://usuario.github.io" \
       -H "Access-Control-Request-Method: POST" \
       -H "Access-Control-Request-Headers: if-match,x-api-key,content-type"
HTTP/1.1 200 OK
access-control-allow-origin: *
access-control-allow-methods: DELETE, GET, HEAD, OPTIONS, PATCH, POST, PUT
access-control-allow-headers: if-match,x-api-key,content-type

$ curl .../catalogo -H "Origin: https://usuario.github.io"
etag: "4812f103051a1d01"
access-control-expose-headers: ETag
```

5.6. Fallos que solo aparecen al desplegar

El sistema funcionaba sin un solo error en la máquina de desarrollo. Al llevarlo a Render aparecieron tres fallos que en local eran **imposibles de observar**, porque ninguno estaba en la lógica del programa: los tres los introducía el entorno de ejecución. Se documentan aquí porque constituyen el resultado más instructivo de la práctica.

El proxy debilita los ETag

El primero inutilizaba por completo el mecanismo central de la práctica. La lectura de un recurso devolvía una etiqueta distinta de la que el servidor había calculado:

Listing 12: La etiqueta que recibe el cliente no es la que emite el servidor

```
ETag calculado por el servidor:   "2a7cce75dc62ea21"
ETag que llega al cliente:      W/"2a7cce75dc62ea21"
```

El prefijo `W/` marca la etiqueta como *validador débil*, y lo añade el proxy de la plataforma por obligación normativa: comprime la respuesta antes de entregarla, de modo que lo que el cliente recibe ya no es idéntico byte a byte a lo que produjo el origen. El RFC 9110 exige que el validador de una representación transformada se marque como débil, precisamente para que nadie la use en comparaciones que requieran identidad exacta.

La consecuencia era que el cliente devolvía `If-Match` con el prefijo, el servidor lo comparaba mediante igualdad literal contra su etiqueta sin prefijo, y **ninguna escritura podía completarse**. Las métricas del servicio de rentas lo registraron con precisión: cinco conflictos y cinco reintentos, todos fallidos, seguidos de un 503 hacia el agregador. La coreografía moría en su tercer paso.

La corrección consiste en comparar la etiqueta opaca, descartando el prefijo. La decisión merece justificarse, porque formalmente `If-Match` exige comparación *fuerte* y una etiqueta débil no debería coincidir nunca: lo que legitima ignorarla aquí es que **la debilidad la introduce la infraestructura y no un cambio de estado del recurso**. El hash sigue siendo el discriminante, y una etiqueta caducada continúa produciendo 412.

El directorio del registro es estado blando

El segundo fallo hacía que el descubrimiento mintiera. El registro respondía con un directorio vacío mientras los tres servicios estaban vivos y atendiendo peticiones.

La causa era un supuesto implícito en el diseño: cada servicio se daba de alta **una sola vez, al arrancar**. Eso basta mientras nada se reinicie, pero el directorio vive en memoria y la plataforma suspende los servicios inactivos. Al despertar, el registro había perdido su diccionario y los demás no tenían ningún motivo para volver a presentarse.

Lo revelador es que el error no estaba en la implementación sino en el **modelo**: tratar un alta como una inscripción permanente. Los registros reales —Eureka, Consul— tratan las altas como **arrendamientos que hay que renovar**, no porque sea más cómodo, sino porque un servicio puede desaparecer sin avisar y el registro no tiene forma de enterarse. La solución adoptada sigue ese modelo: cada servicio se republica al atender su endpoint de gestión, y el agregador, cuando detecta que falta alguien en el directorio, invoca una operación que fuerza el alta y vuelve a leerlo.

Listing 13: El directorio se repara solo, verificado contra el despliegue

```
$ curl .../registro/servicios
{"servicios":[]}

$ curl .../agregador/servicios
  agregador -> https://sd-agregador-iker.onrender.com
  catalogo  -> https://sd-catalogo-iker.onrender.com
  rentas    -> https://sd-rentas-iker.onrender.com
```

Se descartó deliberadamente la alternativa evidente, un latido periódico, porque mantendría despierto al registro de forma permanente y agotaría la cuota de cómputo gratuita. La republicación perezosa aprovecha el tráfico que el servicio ya está atendiendo, de modo que un sistema en reposo no genera tráfico alguno.

La red privada no sirve donde se la esperaba

El tercero habría impedido el despliegue por completo. La configuración inicial resolvía las direcciones entre servicios por la **red privada** de la plataforma, que es lo que recomienda cualquier guía: el tráfico interno no sale a la red pública, ahorra latencia y no se expone. Pero en el plan gratuito un servicio **puede enviar** tráfico privado y **no puede recibirlo**, de modo que el agregador nunca habría alcanzado al catálogo.

El mismo error tenía una segunda cara más sutil: la propiedad que entrega la dirección privada devuelve un par `host:puerto` **sin esquema**, y el cliente HTTP lo rechaza antes de intentar la conexión. Dos fallos distintos en la misma línea de configuración, ninguno de los cuales podía manifestarse en local, donde las direcciones son literales de `localhost`.

Qué tienen en común

Los tres fallos comparten una estructura: el programa era correcto respecto a los supuestos con los que se escribió, y lo que falló fueron **los supuestos**. Que un ETag llega intacto, que un servicio registrado permanece registrado, que una dirección interna es alcanzable. Cada uno es una de las falacias de la computación distribuida que la práctica 5 había enunciado en abstracto, aquí comprobadas en un despliegue real. La diferencia entre escribir un servicio web y **operarlo** es exactamente esta: en local se valida la lógica, pero los supuestos solo se validan desplegando.

6. Conclusiones

Esta práctica cierra el recorrido del curso reconstruyendo, sobre estándares, lo que las anteriores habían resuelto con mecanismos propios. El contraste con la práctica 5 es especialmente revelador: allí se construyó un middleware de objetos remotos que funcionaba, pero que solo entendían los programas escritos para él. Aquí el sistema es consumible desde un navegador, desde `curl` o desde cualquier lenguaje, sin haber escrito una sola línea de cliente específica. **Esa es la aportación de los estándares**: no resuelven problemas que no supiéramos resolver, sino que hacen que la solución sea interoperable.

Sobre la **pila de servicios**, implementarla capa por capa dejó claro que no es una clasificación académica sino una separación de responsabilidades con consecuencias prácticas. La capa de Comunicaciones aportó HTTP y con él un espacio de nombres universal. La de Mensajes obligó a decidir entre aprovechar la semántica del protocolo o tunelizarla. La de Descripción resultó ser la que da valor real al servicio: un servicio sin contrato es inutilizable por terceros, y el hecho de que FastAPI derive el OpenAPI del propio código elimina la fuente habitual de desincronización entre documentación e implementación. La capa de Procesos fue la que justificó tener varios servicios en lugar de uno.

Sobre el **contraste entre SOAP y REST**, implementar ambos sobre el mismo recurso mostró que la diferencia esencial no está en XML frente a JSON, sino en **qué papel se le asigna a HTTP**. SOAP lo usa como un túnel y define su propia semántica dentro del sobre, lo que le da independencia del transporte al precio de volverse opaco: un error se entrega con código 200 y ningún intermediario puede reaccionar a él. REST se apoya en la semántica que HTTP ya tiene, y por eso sus errores, sus cachés y sus precondiciones funcionan con la infraestructura existente.

Sobre la **concurrencia**, la lección es la más valiosa. El mismo problema de las prácticas 3 y 4 —dos clientes que quieren la misma película— se resolvió con la filosofía contraria. Allí se impedía el conflicto bloqueando; aquí se deja ocurrir y se detecta con `ETag` e `If-Match`. Las mediciones mostraron que ninguna de las dos es universalmente mejor: la probabilidad de conflicto resultó ser proporcional a la ventana entre lectura y escritura, hasta el punto de que al bajar la latencia de 3,477 s a 0,018 s los conflictos prácticamente desaparecieron. El enfoque optimista conviene cuando las operaciones son breves; el pesimista, cuando la sección crítica es larga o la contención es alta.

El experimento dejó además un resultado con valor de diseño: el reintento no reduce el número de conflictos, sino que **cambia quién los padece**. Sin reintento el cliente recibió seis errores 412 que no significaban nada para él; con reintento no recibió ninguno, y los únicos rechazos que llegaron fueron 409 honestos que describían una situación real. Un 45 % más de latencia a cambio de que el cliente deje de ver fallos que no son suyos es, en casi cualquier sistema, un buen negocio.

Sobre el **despliegue**, la práctica dejó su lección más inesperada. El sistema no falló por nada que estuviera mal programado, sino por tres supuestos que la máquina de desarrollo nunca puso a prueba: que un `ETag` llega al cliente tal como se emitió, que un servicio dado de alta sigue dado de alta, y que una dirección interna es alcanzable. Los tres se rompieron en la nube, y los tres son casos concretos de las falacias de la computación distribuida que la práctica anterior había enunciado en abstracto. El fallo de los `ETag` es el más elocuente: un intermediario que cumplía escrupulosamente la norma —marcar como débil el validador

de una representación que ha transformado— bastaba para inutilizar el mecanismo completo de concurrencia optimista, sin que una sola línea del programa fuera incorrecta. **En local se valida la lógica; los supuestos solo se validan desplegando.**

Queda por señalar una **limitación** que la práctica no resuelve y que conviene no disimular: el estado vive en la memoria del proceso. Mientras haya una sola instancia de cada servicio el sistema es correcto, pero la propiedad que hace atractivo el estilo sin estado —poder replicar horizontalmente— es justamente la que este diseño no puede aprovechar, porque cada réplica tendría su propio catálogo. Resolverlo exigiría una base de datos compartida, y con ella reaparecerían, en otra capa, los mismos problemas de concurrencia que esta práctica resolvió con `ETag`. Es un recordatorio de que en los sistemas distribuidos los problemas rara vez se eliminan: se desplazan de capa.